

Генераторы списков

Генераторы списков Python

Генераторы списков представляют собой очень мощный и удобный функционал, позволяющий создать список и сразу наполнить его определенными значениями.

Создание генератора списка

Генераторы списка строятся по следующему шаблону:

[выражение for переменная in последовательность]

Генераторы списков Python

Пример 1:

```
a = [0 for i in range(7)]  
print(a)
```

Результат:
[0, 0, 0, 0, 0, 0, 0]

Пример 2:

```
a = [i**2 for i in range(5)]  
print(a)
```

Результат:
[0, 1, 4, 9, 16]



Что получим?

Генераторы списков Python

Пример 3:

 Что получим?

```
a = [i%4 for i in range(1, 10)]  
print(a)
```

Результат:
[1, 2, 3, 0, 1, 2, 3, 0, 1]

Пример 4:

```
a = [i for i in "hello"]  
print(a)  
b = [i*2 for i in "hello"]  
print(b)
```

 Что получим?

Результат:
['h', 'e', 'l', 'l', 'o']
['hh', 'ee', 'll', 'll', 'oo']

Генераторы списков Python

Внутри выражения можно использовать другие функции. Например можно найти код символа в таблице ASCII:

```
a = [ord(i) for i in "abcd"]  
print(a)
```

Результат:
[97, 98, 99, 100]

В качестве выражения могут выступать и функции из модулей.

```
from random import randint  
a = [randint(-10, 10) for i in range(5)]  
print(a)
```

Результат:
[-6, 10, 9, 2, -6]

Генераторы списков Python

Обойдём элементы другого списка при помощи генератора. Возьмём генератор и создадим ещё один, который будет превращать все отрицательные значения в положительные.

```
from random import randint  
a = [randint(-10, 10) for i in range(5)]  
print(a)  
b = [abs(elem) for elem in a]  
print(b)
```

Результат:

[-2, 7, -7, -10, -1]

[2, 7, 7, 10, 1]

Условный оператор в генераторе списка

В генераторах списка можно также использовать условные конструкции. Шаблон в таком случае следующий:

```
[выражение for переменная in  
последовательность if условие]
```

Допустим, из списка *a* в список *b* нужно перенести все элементы, которые являются чётными:

```
from random import randint  
a = [randint(-10, 10) for i in range(5)]  
print(a)  
b = [elem for elem in a if elem%2==0]  
print(b)
```

Результат: [-1, -8, 5, 1, 1]
[-8]

Считывание данных при помощи генератора списка

Рассмотрим пример, когда мы вводим в одну переменную несколько чисел через пробел через `input()`. Получим список чисел с помощью генератора списков:

```
b = [int(num) for num in input().split()]  
print(b)
```

Формат ввода:

1 2 3 4 5

Формат вывода:

[1, 2, 3, 4, 5]



Какая функция позволяет реализовать подобное преобразование?

Вложенные циклы в генераторе списка

В генераторах списка также можно использовать двойные циклы.

```
a = [(i, j) for i in 'abc' for j in [1, 2, 3]]  
print(a)
```

Принцип работы как при обходе обычных вложенных циклов. Мы берём первую переменную из внешнего цикла, и проходим с ней все переменные внутреннего цикла, после чего берётся вторая переменная внешнего цикла и т.д.

Результат:

```
[('a', 1), ('a', 2), ('a', 3), ('b', 1), ('b', 2), ('b', 3), ('c',  
1), ('c', 2), ('c', 3)]
```

Многомерные списки и генераторы списков

Имеем список, элементами которого являются кортежи.
При помощи генераторов списков значения:

```
a = [i*j for i in [2, 3, 4, 5]
      for j in [1, 2, 3] if i*j >= 10]
print(a)
```

Результат:
[12, 10, 15]

 Что получим?

Многомерные списки и генераторы списков

Имеем список, элементами которого являются кортежи. При помощи генераторов списков выберем нужные значения:

```
a = [  
    ("Sidorov", 1995),  
    ("Lukov", 2002),  
    ("Petrov", 1991),  
    ("Gorbachev", 1984)  
]  
  
s = [elem[0] for elem in a]  
y = [elem[1] for elem in a]  
print(s)  
print(y)
```



Что получим?

Результат:

```
['Sidorov', 'Lukov', 'Petrov', 'Gorbachev']  
[1995, 2002, 1991, 1984]
```

Фильтрация с помощью генератора списка

```
a = [("Isaev", 2005),  
      ("Eliseev", 1999),  
      ("Kozlov", 2004),  
      ("Bukov", 1995),  
      ("Gavrilov", 1980),  
      ("Efremov", 2006)
```

```
]
start = [elem[0] for elem in a if  
elem[0].startswith("E")]
```

```
print(start)
```

```
more = [elem[0] for elem in a if elem[1]>2000]
```

```
print(more)
```

```
first= [elem[0][0] for elem in a if elem[1]>2000]
```

```
print(first)
```

Результат:

['Eliseev', 'Efremov']

['Isaev', 'Kozlov', 'Efremov']

['I', 'K', 'E']



Что получим?

Обработка данных словаря при помощи генератора списка

```
a = {  
    'Sidorov': {'age': 1995, 'hobby': 'football', 'car': 'BMW'},  
    'Lukov': {'age': 2002, 'hobby': 'basketball', 'car': 'Opel'},  
    'Petrov': {'age': 1991, 'hobby': 'chess', 'car': 'BMW'},  
    'Gorbachev': {'age': 1984, 'hobby': 'tennis', 'car': 'BMW'}  
}  
  
c = [a[elem]['car'] for elem in a]  
print(c)  
  
h = [a[elem]['hobby'] for elem in a]  
print(h)  
  
n_with_c = [(elem, a[elem]['car']) for elem in a if  
a[elem]['age'] < 2000]  
print(n_with_c)
```



Что получим?

Результат:

['BMW', 'Opel', 'BMW', 'BMW']

['football', 'basketball', 'chess', 'tennis']

[('Sidorov', 'BMW'), ('Petrov', 'BMW'), ('Gorbachev', 'BMW')]

Обработка строки при помощи генератора списка

```
s = 'ry74y543'  
str_digits = [i for i in s if i.isdigit()]  
print(str_digits)
```

```
int_digits = [int(i) for i in str_digits]  
print(int_digits)
```



Что получим?

```
letters = [i for i in s if i.isalpha()]  
print(letters)
```

Результат:

['7', '4', '5', '4', '3']

[7, 4, 5, 4, 3]

['r', 'y', 'y']

Генераторы словарей

Генератор словарей

Генератор словаря необходим для быстрого и удобного создания словаря с наполненными значениями

Общий вид генератора словарей:

```
{ ключ: значение for переменная in  
последовательность }
```

Пример1:

```
a = {i: i**2 for i in range(1, 5)}  
print(a)
```



Что получим?

Результат:

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

Генератор словарей

Внутри генератора словарей можно обходить любую коллекцию, поддерживающую итерацию.

Например:

```
lst = ['hi', 'hello', 'www']  
a = {word: len(word) for word in lst}  
print(a)
```



Что получим?

Результат:

```
{'hi': 2, 'hello': 5, 'www': 3}
```

Генератор словарей

Пример 3:

```
users = [  
    [0, 'Bob', 'password'],  
    [1, 'code', 'python'],  
    [2, 'Stack', 'overflow'],  
    [3, 'username', '1234'],  
    [51, 'qwerty', '1234']]  
new_users = {user[0]:user[1:] for user in users}  
print(new_users)
```

Результат:

```
{2: ['Stack', 'overflow'],  
 3: ['username', '1234'],  
51: ['qwerty', '1234']}
```



Что получим?

Генератор словарей

При помощи генератора можно обходить и уже существующие словари.

Например:

```
data = {  
    'Джефф Безос': '177',  
    'Илон Маск': '126',  
    'Бернар Арно': '150',  
    'Билл Гейтс': '124'}
```

```
new_data = {key.title(): int(value) for key,  
value in data.items()}  
print(new_data)
```

Результат:

```
{'Джефф Безос': 177,  
'Илон Маск': 126,  
'Бернар Арно': 150,  
'Билл Гейтс': 124}
```



Что получим?

Генераторы множеств

Генератор множества

Пусть требуется создать множество, содержащее цифры введенного числа.

Попробуем использовать следующий код:

```
digits = set(int(input()))  
print(digits)
```



Получим
нужный
результат?

Результат:

TypeError: 'int' object is not iterable

При решении данной задачи удобно использовать генератор множества:

```
digits = {int(c) for c in input()}  
print(digits)
```

Формат ввода: 123

Формат вывода: {1, 2, 3}

Генератор множества

Общий вид генератора множеств следующий:

```
{выражение for переменная in  
последовательность }
```

Например:

```
digits = {int(d) for d in 'abcd12e0' if  
d.isdigit() }  
print(digits)
```



Что получим?

Результат:
{0, 1, 2}

Выражения-генераторы

Выражения - генераторы

В Python можно создавать итераторы с помощью генераторов, которые делятся на два типа:

- ❑ выражения генераторы
- ❑ функции генераторы

Рассмотрим выражения – генераторы:

```
b = (i**2 for i in range(1, 6))  
print(b)
```

Результат:

<generator object <genexpr> at 0x7f8682d94270>



Переменная `b` является объектом типа генератор.



Что получили?

Выражения - генераторы

Генератор (объект специального типа `<class 'generator'>`) – это коллекция, по которой можно итерироваться только один раз.

Под **итерацией** подразумевается процесс перебора элементов коллекции внутри цикла или функции, которая делает перебор.

Таким образом **генератор** реализует протокол итератора, т.е. **является итератором**, значит к нему можно вызывать функцию `next`:



Что получим?

```
b = (i**2 for i in range(6, 16))  
print(next(b))  
print(next(b))
```

Результат:

36

49

Выражения - генераторы

Работа с генератором происходит по стандартному сценарию работы с итератором.

Можно:

- ☐ вызывать функцию `next()` для получения очередного значения генератора
- ☐ итерироваться с помощью цикла `for` по генератору
- ☐ распаковывать генератор
- ☐ проверять принадлежность с помощью оператора `in`

Выражения - генераторы

Обойдем элементы генератора в цикле for:

```
b = (i**2 for i in range(5, 7))  
for i in b:  
    print(i)  
for i in b:  
    print(i)
```



Что получим?

Результат:

25

36



Обойти элементы генератора при помощи цикла можно только один раз!

Выражения - генераторы

Рассмотрим итерацию при вызове функции sum:

```
b = (i**2 for i in range(1, 4))  
print(sum(b))  
print(sum(b))
```



Что получим?

Результат:

14

0



При втором выводе получен 0, поскольку по элементам генератора b уже проитерировалась первая функция sum.

Операции, которые выражения-генераторы не поддерживают

Ограничения генераторов:

- ❑ нельзя получить длину генератора функцией `len()`
- ❑ нельзя распечатать элементы генератора функцией `print()` без предварительной распаковки
- ❑ у генератора нельзя получить элемент по индексу
- ❑ после итерации по генератору он становится пустым

Функция генератор.

Функция генератор

Функция генератор – это функция, которая возвращает объект специального типа `<class 'generator'>`, который является итератором.

Вместо ключевого слова **return** в функции генераторе используется другое ключевое слово – **yield**

Например:

```
def gen_func():  
    for i in [1, 2, 3]:  
        yield i  
  
f = gen_func()  
print(next(f))  
print(next(f))
```



Что получим?

Результат:

1

2

Пространство имен функции генератора

Когда вызывается
обычная функция



Она получает личное пространство имен, в котором создаются ее локальные переменные.

Когда обычная
функция достигает
оператора `return`



Локальные переменные уничтожаются и значение возвращается вызывающей стороне.

Последующий вызов
той же функции



Создает новое локальное пространство имен и новый набор локальных переменных.



Функция генератор, напротив сохраняет локальные переменные от вызова к вызову. Это своего рода **возобновляемая функция**.

Функция генератор

Может показаться, что в теле функции генератора всегда должен быть цикл, но это необязательно.

Например:

```
def generate_1234():  
    yield 1  
    yield 2  
    yield 3  
    yield 4  
  
f = generate_1234()  
print(next(f))  
print(next(f))  
print(next(f))
```



Что получим?

Результат:

1
2
3

Функция генератор с дополнительными действиями

Функция генератор может не только порождать значения, но и совершать различные побочные действия во время выполнения, такие как:

- ☐ вывод текста на экран
 - ☐ запись данных в файл
 - ☐ приостановка исполняющейся программы на некоторое время
- и т.д.

Функция генератор с дополнительными действиями

Приведенный ниже код:

```
def generate_AB():  
    print('start')  
    yield 'A'  
    print('continue')  
    yield 'B'  
    print('end')  
  
for char in generate_AB():  
    print('-->', char)
```

Выводит:

start

--> A

continue

--> B

end

Ключевое слово `return` в теле функции генератора

До версии Python 3.3 наличие ключевого слова `return` внутри функции генератора приводило к возникновению ошибки.

Теперь это допускается, при этом инструкция `return` приводит к возбуждению исключения **StopIteration**.

Приведенный ниже код:



Что получим?

```
def generate_ints():
```

```
    yield 1
```

```
    yield 2
```

```
    return 3
```

```
    yield 4
```

```
for num in generate_ints():
```

```
    print(num)
```

Выводит:

1

2



Само
значение 3 не
выводится!

Примеры использования функций генераторов

Пример 1. Реализуем функцию генератор `counter()`, которая возвращает генератор, порождающий последовательность целых чисел значениями от `low` до `high` включительно с шагом один.



Что получим?

```
def counter(low, high):  
    for num in range(low, high + 1):  
        yield num  
  
counter1 = counter(100, 106)  
print(next(counter1))  
print(next(counter1))
```

Результат:
100
101

Примеры использования функций генераторов

Пример 2. Реализуем функцию генератор `even_numbers()`, которая возвращает генератор, порождающий бесконечную последовательность целых четных чисел от значения `begin`.

```
def even_numbers(begin):  
    while True:  
        yield begin  
        begin += 2  
  
evens1 = even_numbers(10)  
for index, num in enumerate(evens1):  
    if index > 2:  
        break  
    print(num)  
  
evens2 = even_numbers(101)  
print(next(evens2))  
print(next(evens2))
```



Что получим?

Результат:

10

12

14

101

103

Примеры использования функций генераторов

Пример 3. Реализуем функцию генератор `wrapper`, которая возвращает генератор, порождающий последовательность символов строки `text`, обрамленных специальным символом `symbol`.

```
def wrapper(text, symbol):  
    for char in text:  
        yield symbol + char + symbol  
  
s1 = wrapper('go', '/')  
for char in s1:  
    print(char)  
  
s2 = wrapper('Python', '+')  
print(next(s2))  
print(list(wrapper('cat', '-')))
```



Что получим?

Результат:

/g/

/o/

+P+

['-c-', '-a-', '-t-']

Конструкция `yield from`

Рассмотрим генераторную функцию `get_data`:

```
def get_data():  
    for num in range(3):  
        yield num  
    for char in 'ABC':  
        yield char  
for i in get_data():  
    print(i)
```

Результат:

0

1

2

A

B

C



Что получим?

Конструкция `yield from`

Генераторную функцию `get_data()` можно упростить, если использовать синтаксическую конструкцию **`yield from`** `<iterable>`, которая появилась в Python 3.3.

```
def get_data():  
    yield from range(3)  
    yield from 'ABC'  
for i in get_data():  
    print(i)
```

Результат:

0

1

2

A

B

C

Синтаксическая конструкция `yield from` `<iterable>` позволяет объединить две конструкции: `yield` и цикл `for`.

Конструкция `yield from`

Реализуем генераторную функцию `chain(*iterables)`, которая принимает произвольное количество итерируемых объектов и возвращает генератор

```
def chain(*iterables):  
    for it in iterables:  
        yield from it  
for i in chain('AB', [1, 2], (4, 5),  
               {'name': 'Timur', 'age': 29}):  
    print(i, end=' ')
```



Что получим?

Результат:

A B 1 2 4 5 name age

Конструкция `yield from`

Конструкция `yield from` позволяет вкладывать один генератор в другой, таким образом создавать субгенераторы (вложенные генераторы).

```
def generator2():  
    yield 'Red'  
    yield 'Blue'  
def generator1():  
    yield 'Green'  
    yield from generator2()  
    yield 'Yellow'  
for color in generator1():  
    print(color, end=' ')
```



Что получим?

Результат:
Green Red Blue Yellow

Обработка исключений

Типы ошибок

Типы ошибок

Разработка программы на любом языке программирования практически всегда бывает связана с возникновением различного рода ошибок, препятствующих получению желаемого результата.

Выделяют следующие три категории ошибок:

- ❑ **синтаксические** — возникают из-за синтаксических погрешностей кода
- ❑ **логические** — проявляются вследствие логических неточностей в алгоритме
- ❑ **исключения, ошибки времени выполнения** — вызваны некорректными действиями пользователя или системы

Синтаксические ошибки

Синтаксические ошибки являются следствием несоблюдения общепринятого синтаксиса языка. Другими словами, это ошибки, связанные с неправильно набранным кодом.

Например:

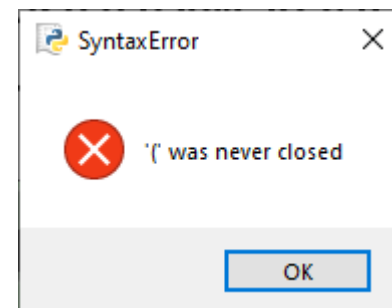
```
print('Hello, world! '
```



Какая ошибка допущена?

Допущена синтаксическая ошибка: пропущена закрывающая скобка. При попытке его запуска получим:

SyntaxError: '(' was never closed



Синтаксические ошибки

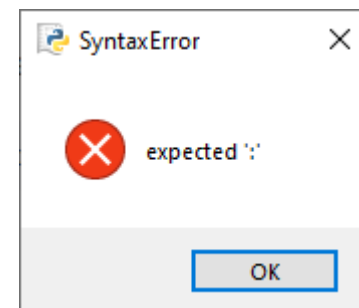
Например:

```
def square(num)
    return num ** 2
```



Какая ошибка допущена?

Допущена синтаксическая ошибка: **пропущен символ двоеточия** в описании функции. При попытке его запуска получим: `SyntaxError: expected ':'`



Синтаксические ошибки легко отлавливаются интерпретатором, который сразу же сообщает программисту о проблеме в написанном коде.

Логические ошибки

Логические ошибки считаются более сложными в выявлении, поскольку не отлавливаются интерпретатором.

Обычно они вызваны определенным недостатком в логике программы, из-за чего результат работы программы отличается от желаемого результата.

Возможным решением проблемы является тестирование программы на разных примерах входных данных, для которых известен правильный результат.

Логические ошибки

Например функция `avg()`, подсчитывающая среднее значение переданных в нее аргументов:

```
def avg(a, b):  
    return a + b / 2  
print(avg(0, 7))  
print(avg(6, 14))
```

Выводит:
3.5
13.0



Допущена ли
ошибка?

В теле функции допущена **логическая** ошибка, пропущены скобки, на два должна делиться сумма чисел `a` и `b`, а не только `b`.



Логические ошибки могут проявлять себя только при определенных условиях.

Исключения

Исключительные ситуации или исключения (exceptions) – это ошибки, обнаруженные при исполнении.

Исключения проявляются в зависимости от наличия обстоятельств, меняющих ход выполнения программы, связаны с некорректностью переданных в программу данных, недоступностью ресурсов и т.д.

Например, к чему приведет попытка чтения несуществующего файла? Или если файл был случайно удален пока программа работала? Такие ситуации обрабатываются при помощи исключений.

Исключения

Например:

```
num1 = 10  
num2 = 0  
print(num1 / num2)
```



Что получим?

Выводит:
ZeroDivisionError:
division by zero

Деление на ноль провоцирует исключительную ситуацию, которая приводит к аварийному завершению работы и выводу ошибки на экран. ZeroDivisionError — это название исключения, а division by zero — его краткое описание.



При обнаружении необработанного исключения Python немедленно останавливает выполнение программы и выводит сообщение об ошибке!

Работа с кодами возврата

В процедурном программировании синтаксис работы с исключениями был прост и основывался на том, что вернула функция. Если функция возвращала `True` — все хорошо, если же `False` — то произошла ошибка.

При этом существовало два подхода к работе с ошибками:

- ❑ **Подход два в одном** — функция возвращает `False` как для ожидаемой, так и для неожиданной ошибки.
- ❑ **Подход разделения ошибок**, при котором функция возвращает `False` в случае неожиданной ошибки, а ожидаемую ошибку возвращает отдельным возвращаемым значением (числовым).

Работа с кодами возврата

У работы с исключениями есть преимущества перед кодами возврата. В современных языках программирования, таких как Python, C#, Java, GO и т.д., используются именно исключения.

Несмотря на то что язык Python полностью поддерживает работу с исключениями, как с полноценными объектами, мы все же можем встретить следы работы с кодами возврата, которые Python унаследовал от языка C.

Работа с кодами возврата

Например: Строковый тип данных `str` содержит два похожих метода: `find()` и `index()`.



Для чего используются эти методы?

Оба метода ищут позицию первого вхождения подстроки в заданную строку.

Следующий программный код:

```
text = 'Hello, world!'
print(text.find('w'))
print(text.find('a'))
print(text.index('a'))
```

Выводит:

7

-1

ValueError: substring not found

Метод `find()` использует механизм кодов возврата



Что получим?

Метод `index()` возбуждает исключение

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
product = 1
for i in range(1, 11):
    product *= i
print('Произведение чисел от 1 до 10
      включительно:', product)
```

Синтаксическая ошибка



Тип ошибки?

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
total = 1
for i in range(1, 11):
    total += i
print('Сумма всех чисел от 1 до 10
      включительно:', total)
```

Логическая ошибка



Тип ошибки?

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
ages = { 'Mickey': 93,  
         'Donald': 87 ,  
         'Goofy': 89 }  
print (ages [ 'Donald' ] )
```

Синтаксическая ошибка



Тип ошибки?

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
def div(a, b):  
    return a / b  
print(div(7, 0))
```

Исключение



Тип ошибки?

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
def div(a, b):  
    if b = 0:  
        return None  
    return a / b  
print(div(7, 0))
```

Синтаксическая ошибка



Тип ошибки?

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
matrix = [[1, 2, 3],  
          [4, 5, 6],  
          [7, 8, 9]]  
  
for row in matrix:  
    for elem in row:  
        print(elem)  
  
print()
```

?

Тип ошибки?

Синтаксическая ошибка

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7]  
print(numbers[7])
```

Исключение



Тип ошибки?

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
def add(a, b):  
    return a + b  
print(add('bee', 10))
```



Тип ошибки?

Исключение

Складывать можно только
объекты одного типа!

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
ages = {'Mickey': 93,  
        'Donald': 87,  
        'Goofy': 89}  
print(ages['Timur'])
```



Тип ошибки?

Исключение

Ревью кода

Найдите ошибку, которую содержит приведенный ниже код:

```
product = 0
for i in range(1, 11):
    product *= i
print('Произведение чисел от 1 до 10
      включительно:', product)
```

Логическая ошибка



Тип ошибки?

Обработка исключений

Основные типы исключений

Обработка исключений

Общая идея:

Программный код, в котором теоретически может возникнуть ошибка, выделяется специальным образом – "берется на контроль".

Если при выполнении этого программного кода ошибка не возникает, то ничего не происходит.

Если при выполнении "контролируемого" кода возникает ошибка, то выполнение кода останавливается и автоматически создается объект-исключение, содержащий описание возникшей ошибки.

Обработка исключений

Для обработки исключительных ситуаций в языке Python используется конструкция **try-except**.

После ключевого слова **try** и двоеточия размещается блок программного кода, который мы подозреваем на предмет возможного возникновения ошибки.

Этот код называется **контролируемым**.

По завершении этого блока указывается ключевое слово **except** (с двоеточием), после которого идет еще один блок программного кода.

Этот код называется **кодом обработки ошибки (исключения)**.

Обработка исключений

Шаблон конструкции обработки исключений:

```
try:  
    # контролируемый код  
except:  
    # код обработки ошибки (исключения)
```

Если при выполнении кода в блоке try ошибка не возникла, то код обработки ошибки в блоке except выполняться не будет.

Если при выполнении кода в блоке try возникла ошибка, то выполнение кода try блока прекращается, и выполняется код обработки ошибки (исключения) в блоке except. После этого управление передается следующей команде после конструкции try-except.

Обработка исключений

Пример использования инструкции try-except:

try:

```
num1 = int(input())
```

```
num2 = int(input())
```

```
print('Частное чисел равно', num1 / num2)
```

except:

```
print('Вы ввели некорректные данные!')
```

Протокол ввода 1:

10

2

Протокол вывода 1:

Частное чисел равно 5.0

При выполнении
кода в блоке try
ошибка не
возникла

Обработка исключений

Пример использования инструкции try-except:

try:

```
num1 = int(input())
```

```
num2 = int(input())
```

```
print('Частное чисел равно', num1 / num2)
```

except:

```
print('Вы ввели некорректные данные!')
```

Протокол ввода 2:

abc

Протокол вывода 2:

Вы ввели некорректные данные!

При выполнении
кода в блоке try
возникла ошибка

Обработка исключений

Пример использования инструкции try-except:

try:

```
num1 = int(input())
```

```
num2 = int(input())
```

```
print('Частное чисел равно', num1 / num2)
```

except:

```
print('Вы ввели некорректные данные!')
```

Протокол ввода 3:

10

0

Протокол вывода 3:

Вы ввели некорректные данные!

При выполнении
кода в блоке try
возникла ошибка

Обработка исключений

Пример использования инструкции try-except:

```
try:
```

```
    num1 = int(input())
```

```
    num2 = int(input())
```

```
    print('Частное чисел равно', num1 / num2)
```

```
except:
```

```
    print('Вы ввели некорректные данные!')
```



Такая схема обработки ошибок (исключений), проста и удобна. Но недостаток ее в том, что **разные ошибки (исключения) обрабатываются одинаково**. То есть, какая бы ни возникла ошибка (исключение), реакция программы будет одна и та же.

Обработка нескольких исключений

Обработка нескольких исключений

При обработке нескольких исключений, обработка базируется на **типе или характере ошибки**.

В конструкции try-ехсепт после блока try указывается несколько ехсепт блоков, причем для каждого блока явно указывается тип ошибки (исключения), который обрабатывается в этом блоке.

Ключевое слово, определяющее тип ошибки, указывается после ключевого слова ехсепт соответствующего блока.

Обработка нескольких исключений

Шаблон конструкции обработки нескольких исключений:

```
try:
    # контролируемый код
except тип_ошибки_1:
    # код обработки ошибки (исключения)
except тип_ошибки_2:
    # код обработки ошибки (исключения)
...
except тип_ошибки_n:
    # код обработки ошибки (исключения)
```

Обработка нескольких исключений

Например:

```
try:
    num1 = int(input())
    num2 = int(input())
    print('Частное чисел равно', num1 / num2)
except ValueError:
    print('Нужно было ввести числа!')
except ZeroDivisionError:
    print('На ноль делить нельзя!')
```



В данном случае обрабатываются только ошибки типов `ValueError` и `ZeroDivisionError`. Если бы теоретически возникла ошибка какого-то иного типа, то она бы перехвачена не была!

Обработка нескольких исключений

Для нескольких типов ошибок можно создать один `except` блок:

```
try:
    num1 = int(input())
    num2 = int(input())
    print('Частное чисел равно', num1 / num2)
except (ValueError, IndexError, KeyError):
    print('Обрабатываются три типа ошибок!')
except ZeroDivisionError:
    print('На ноль делить нельзя!')
except:
    print('Если не сработал ни один из
          предыдущих блоков except.')
```

Последний `except` обрабатывает любой тип исключения, отличный от указанных выше, причем такой блок должен быть только один, иначе возникнет ошибка `SyntaxError`.



Основные типы исключений

Тип исключения	Описание
IndexError:	Возникает, когда индекс (например, для элемента списка) указан неправильно (выходит за границы допустимого диапазона)
KeyError:	Возникает при неверно указанном ключе словаря
NameError:	Возникает, если не удастся найти переменную с некоторым именем
SyntaxError:	Возникает при наличии в исходном коде синтаксических ошибок

Основные типы исключений

Тип исключения	Описание
TypeError:	Возникает при несоответствии типов, когда для обработки требуется значение определенного типа, а передается значение другого типа
FileNotFoundError:	Возникает при открытии несуществующего файла
ValueError:	Возникает, когда в функцию передается аргумент с неподдерживаемым значением
ZeroDivisionError:	Возникает при попытке выполнить деление на ноль

Обработка исключений

Обработчики исключений обрабатывают исключения не только в том случае, если они возникают непосредственно в блоке **try**, но и в том случае, если они возникают внутри функций, вызываемых в блоке **try**.

Например:

```
def this_fails():  
    num = 1 / 0  
  
try:  
    this_fails()  
except ZeroDivisionError:  
    print('Деление на ноль')
```

Результат:
Деление на ноль

Необязательный блок else

Блок **else** размещается после последнего except блока и содержит программный код, который выполняется только в том случае, если при выполнении кода в try блоке **ошибок (исключений) не было**.

Шаблон инструкции try-except в этом случае такой:

```
try:
    # контролируемый код
except тип_ошибки_1:
    # код обработки исключения
...
except тип_ошибки_n:
    # код обработки исключения
else:
    # код для случая, если ошибки не было
```

Необязательный блок else

Рассмотрим программный код:

```
try:
    num = int(input())
    print('Квадрат числа равен:', num ** 2)
except ValueError:
    print('Вы ввели некорректные данные!')
else:
    print('Ошибки не произошло!')
```

Протокол ввода 1:

10

Протокол вывода 1:

Квадрат числа равен: 100

Ошибки не произошло!

Необязательный блок else

Рассмотрим программный код:

```
try:
    num = int(input())
    print('Квадрат числа равен:', num ** 2)
except ValueError:
    print('Вы ввели некорректные данные!')
else:
    print('Ошибки не произошло!')
```

Протокол ввода 2:

abc

Протокол вывода 2:

Вы ввели некорректные данные!

Необязательный блок `finally`

Блок `finally` размещается после последнего `except` блока, либо после блока `else`, если он присутствует, и содержит программный код, который **выполняется в любом случае**, независимо от того, возникла ошибка (исключение) при выполнении кода `try` блока или нет.

```
try:
    # контролируемый код
except тип_ошибки_1:
    # код обработки ошибки (исключения)
...
except тип_ошибки_n:
    # код обработки ошибки (исключения)
finally:
    # код, который выполняется всегда
```

Необязательный блок finally

Рассмотрим программный код:

```
try:
    file = open('data.txt', encoding='utf-8')
    try:
        text = file.read()
    except:
        print('При чтении из файла произошла ошибка!')
    else:
        print('Чтение из файла прошло успешно!')
    finally:
        file.close()
except FileNotFoundError:
    print('Файл с указанным именем не найден!')
```



Блоки try-except можно вкладывать один в другой. Python не ограничивает в количестве вложений.

Необязательный блок **finally**

При наличии ошибки (исключения) и отсутствии блока **except**, инструкции внутри блока **finally** будут выполнены, но ошибка (исключение) не будет обработана.

```
try:  
    x = 10 / 0  
finally:  
    print('Блок finally')
```

Результат:

Блок finally

Traceback (most recent call last):

File "main.py", line 2, in <module>

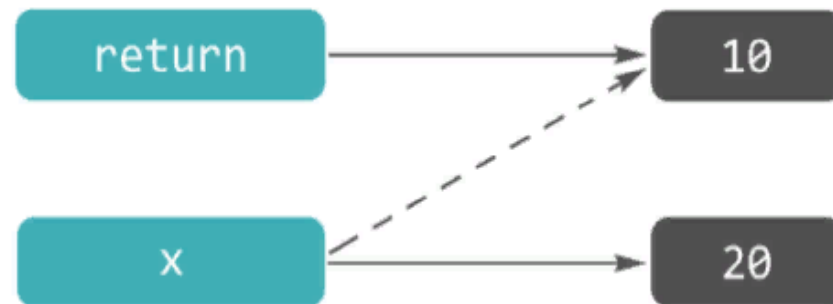
x = 10 / 0

ZeroDivisionError: division by zero

Необязательный блок finally

Рассмотрим следующий код:

```
def f():  
    try:  
        x = 10  
        return x  
    finally:  
        x = 20  
print(f())
```



Что получим?

Результат:
10



Оператор return сохраняет значение возвращаемой переменной, поэтому дальнейшее присвоение переменной x нового значения не повлечет за собой изменения возвращаемого функцией значения.

Необязательный блок finally

Рассмотрим следующий код:

```
def f():  
    try:  
        x = [10]  
        return x  
    finally:  
        x.append(20)  
print(f())
```



Что получим?

Результат:
[10, 20]

Иерархия исключений

В Python присутствует строгая иерархия исключений, вершиной которой является тип `BaseException`.

Тип `BaseException` является классом самого верхнего уровня и базовым для всех прочих классов исключений.

Тип `Exception` – базовый класс для **встроенных** в Python исключений. Именно его, необходимо наследовать при создании пользовательского класса исключения.

Рассмотрим дерево встроенных исключений.

Исключения `IndexError` и `KeyError` являются потомками (наследниками) какого исключения?

`LookupError`

Какие исключения являются потомками (наследниками) исключения `ArithmeticError`?

`FloatingPointError`
`OverflowError`
`ZeroDivisionError`

Иерархия исключений

Для проверки родства классов исключений можно использовать встроенную функцию `issubclass()`, которая указывает на то, является ли некоторый класс потомком указанного класса или нет.



Что получим?

```
print(issubclass(IndexError, LookupError))
print(issubclass(KeyError, Exception))
print(issubclass(ZeroDivisionError,
                  ArithmeticError))
print(issubclass(IndexError,
                  ArithmeticError))
print(issubclass(FloatingPointError,
                  OverflowError))
```

Результат:
True
True
True
False
False

Иерархия исключений

Если у нас используется несколько блоков `except`, то первыми нужно указывать наиболее конкретные, т. к. обработчик исключений может поймать не только указанные типы исключений, но и всех их потомков.

```
try:
    nums = [10, 5, 20, 25]
    print(nums[100])
except Exception:
    print('Произошла ошибка!')
except (KeyError, IndexError):
    print('Ошибка связанная с индексом!')
```



Что плохо?



Здесь первый блок `except` отлавливает вообще все исключения, ведь любое конкретное исключение наследуется от типа `Exception`. Второй блок `except` не имеет шанса хоть раз быть выполненным!

Работа с объектом возбужденного исключения

Если нужен доступ к сгенерированному исключению как к объекту, то используется специальный синтаксис:

```
try:
    nums = [10, 5, 20, 25]
    print(nums[100])
except (KeyError, IndexError) as err:
    # записываем исключение в переменную err
    print(err)
    print(type(err))
```

Результат:
list index out of range
<class 'IndexError'>



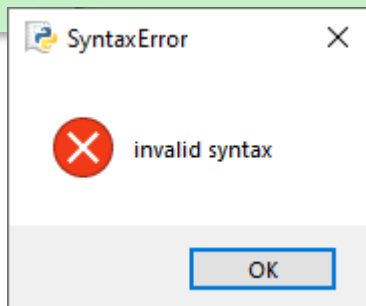
В имени **err** нет ничего особенного. Можно использовать любое имя по своему выбору. Обычно используют имена **err**, **exp**, **e**.

Работа с объектом возбужденного исключения

Определите какой код синтаксически верен:

1

```
try:  
    x = 1 / 0  
except as err:  
    print(err)
```



```
try:  
    x = 1 / 0  
except Exception as err:  
    print(err)
```

2

Результат:
division by zero



Если при обработке всех исключений одним блоком эксерт мы хотим получить доступ к объекту исключения, то нужно явно указать его тип.

Методики LBYL и EAFP

При написании программного кода, который может потенциально возбуждать исключения, существуют два основных подхода:

LBYL (Look Before You Leap) — посмотри перед прыжком

EAFP (Easier to Ask Forgiveness than Permission) — проще извиниться, чем спрашивать разрешение

Приведенный ниже код

```
data = {'Timur': 29, 'Ivan': 54}
data['Anri'] += 1
```

приводит к возникновению исключения `KeyError`, поскольку ключ `Anri` отсутствует в словаре.



Как исправить код?

Методики LBYL и EAFP

Можно исправить такой код двумя способами.

```
data = {'Timur': 29, 'Ivan': 54}
if 'Anri' in data:
    data['Anri'] += 1
else:
    print('Ключ Anri отсутствует в словаре.')
```

1

Перестраховаться и заранее проверить, что все получится. Это идеология **LBYL-подхода**. Сначала посмотрели, убедились, что все в порядке, только потом сделали.

Методики LBYL и EAFP

Можно исправить такой код двумя способами.



```
data = {'Timur': 29, 'Ivan': 54}
try:
    data['Anri'] += 1
except KeyError:
    print('Ключ Anri отсутствует в словаре.')
```

Мы можем описывать только главный алгоритм, рассчитывая, что все будет хорошо. Но при таком подходе необходимо прописать действия с исключениями (иногда разных типов). Это суть подхода **EAFP**.

Оператор `assert`

Оператор `assert` позволяет писать проверки работоспособности кода. Эти проверки называют **утверждениями**. Используются такие утверждения для того чтобы убедиться, остаются ли верными определенные условия во время разработки программы.

Если какое-либо из утверждений оказывается ложным, то это означает, что в программе есть ошибка. Если все утверждения истинны, то в программе ошибок нет.

После того как мы отладили и протестировали наш код с помощью утверждений, мы можем их отключить.



Утверждения могут сделать код более эффективным, устойчивым и надежным.

Оператор `assert`

Оператор `assert` — это встроенный оператор используемый для проверки того, является ли заданное утверждение истинным или ложным.

Если утверждение истинно, то ничего не происходит и выполняется следующая строка кода.

Если же утверждение ложно, оператор `assert` останавливает выполнение программы и подобно оператору `raise` возбуждает исключение `AssertionError`.

Оператор assert

Синтаксис использования оператора assert следующий:

```
assert <утверждение>
```

Также можно добавить дополнительное сообщение, которое описывает подробности возникновения исключительной ситуации. В таком случае синтаксис оператора assert будет следующий:

```
assert <утверждение>, <сообщение>
```

Примеры использования оператора `assert`

Пример 1.

```
age = -29    # возраст человека  
assert age > 0, 'Возраст – положительное число!'
```

Результат:

AssertionError: Возраст – положительное число!

Переменная `age` имеет отрицательное значение, поэтому оператором `assert` возбуждено исключение `AssertionError`, вместе с добавленным сообщением.

Примеры использования оператора `assert`

Пример 2.

```
num1 = 20
num2 = 0
assert num2 != 0, 'Делитель равен нулю.'
print('Частное равно:', num1 / num2)
```

Результат:

AssertionError: Делитель равен нулю.

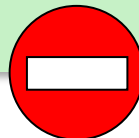
Утверждение `num2 != 0` ложно, поэтому оператором `assert` будет возбуждено исключение `AssertionError` вместе с добавленным сообщением.

Оператор assert

Определите, какой код синтаксически верен:

1

```
assert (2 + 2 == 5, "We've got a problem")
```



2

```
assert 2 + 2 == 5, "We've got a problem"
```



!

assert – это именно оператор, а не функция. При использовании assert мы не указываем круглые скобки!

Оператор assert

Основные тезисы относительно оператора assert:

- ❑ оператор assert — это средство отладки, которое проверяет утверждение, выступающее в качестве внутренней самопроверки вашей программы
- ❑ оператор assert должен применяться только для того чтобы помогать разработчикам идентифицировать ошибки. Он не является механизмом обработки исключений
- ❑ оператор assert может быть глобально отключен в настройках интерпретатора

Возбуждение исключений

Возбуждение исключений

Для возбуждения исключения используется оператор **raise**.

В качестве аргумента оператор **raise** использует экземпляр класса, унаследованного от `Exception`.

Класс указывает на тип исключения, а аргумент, передаваемый конструктору этого исключения, обычно описывает подробности возникновения исключительной ситуации.

Возбуждение исключений

Если возбуждаемые пользователем исключения не перехватываются, то они распространяются вплоть до стандартного обработчика исключений и прекращают работу программы с выводом стандартного сообщения об ошибке.



Возбуждаемые пользователем исключения перехватываются тем же способом, что и исключения, которые возбуждает интерпретатор Python.

Возбуждение исключений

Приведенный ниже код:

Возбуждение исключения вручную

```
try:
    raise IndexError('Ошибка в коде')
except Exception as err:
    print(err)
    print(type(err))
```

ВЫВОДИТ:

```
Ошибка в коде
<class 'IndexError'>
```

Возбуждение исключений

Пример 2:

```
def get_month_name(index):  
    try:  
        if not str(index).isdigit():  
            raise TypeError('Аргумент должен быть  
                               числом.')  
        if index < 1 or index > 12:  
            raise ValueError('Аргумент должен быть  
                               числом от 1 до 12.')  
    except Exception as err:  
        return err  
    else:  
        return index  
  
print(get_month_name(25))  
print(get_month_name('строка'))  
print(get_month_name(8))
```

Результат:

Аргумент должен быть
числом от 1 до 12.

Аргумент должен быть
числом.

8

Возбуждение исключений

При создании объекта исключения мы можем указать в конструкторе подробности возникновения исключительной ситуации.

Конструктор любого исключения принимает переменное количество аргументов.

Для того чтобы получить указанные значения, мы используем атрибут **args**, который является кортежем.

Возбуждение исключений

Приведенный ниже код:

```
try:
    x, y = 10, 0
    if y == 0:
        raise ZeroDivisionError('Произошло
                                деление на ноль.')
except ZeroDivisionError as err:
    print(err)
    print(err.args)
    print(type(err.args))
```

Подробности возникновения
исключительной ситуации

ВЫВОДИТ:

```
Произошло деление на ноль.
('Произошло деление на ноль.',)
<class 'tuple'>
```



При печати объекта
исключения
автоматически
происходит вызов
функции str().

Возбуждение исключений

Оператор возбуждения исключений `raise` имеет несколько вариантов формата:

Вариант 1:

`raise <экземпляр класса>`

В данном формате оператора `raise` указывается экземпляр класса возбуждаемого исключения с аргументами при необходимости.

Пример:

```
raise ValueError('Описание исключения')
```

Возбуждение исключений

Вариант 2:

`raise <название класса>`

Во втором варианте оператора `raise` указывается только название класса исключения.

Пример:

```
raise ValueError  
# эквивалентно: raise ValueError()
```

Примечания

Примечание 1:

Механизм исключений достаточно медленный, поэтому, например, не очень хорошей идеей будет возбуждать исключение внутри цикла, когда мы точно знаем, что их будет достаточно большое количество.

Примечания

Примечание 2:

При передаче переменной с исключением в функцию `print()`, если аргументов было передано несколько, они будут выведены все в виде кортежа.

Приведенный ниже код:

```
try:
    raise ValueError('Ой', 'Произошла ошибка')
except ValueError as e:
    print(e)
```

ВЫВОДИТ:

```
('Ой', 'Произошла ошибка')
```

Возбуждение исключений

Что покажет приведенный ниже код?

```
data = {'Chrome': 'browser', 'Discord':  
'messenger', 'Steam': 'digital distributor'}  
try:  
    try:  
        app_name = 'Firefox'  
        app_type = data[app_name]  
    except:  
        raise KeyError('Not found', app_name)  
except KeyError as e:  
    print(e.args)
```

Результат:
('Not found', 'Firefox')

Пользовательские исключения

В Python можно создавать собственные типы исключений. Такая практика позволяет увеличить гибкость процесса обработки ошибок в рамках той предметной области, для которой написана программа.

Для создания собственного типа исключения необходимо создать класс, являющийся потомком (наследником) одного из уже существующего типа исключения. Самым верным вариантом является класс `Exception`.

Пользовательские исключения

Приведенный ниже код:

```
class NegativeAgeError(Exception) :  
    pass
```

Описывает новый тип исключения с именем `NegativeAgeError`, который является потомком класса `Exception`.

Класс `Exception` содержит весь необходимый функционал, позволяющий работать с исключениями, поэтому в большинстве случаев достаточно создать пустой класс, который является потомком класса `Exception`.



Теперь мы можем работать с типом исключения `NegativeAgeError`, как с любым встроенным.

Пользовательские исключения

Например:

```
class NegativeAgeError(Exception):  
    pass  
  
try:  
    print('Введите свой возраст:')  
    age = int(input())  
    if age < 0:  
        raise NegativeAgeError('Возраст не может  
                                быть отрицательным')  
    print('Ваш возраст равен', age)  
except ValueError:  
    print('Возраст должен быть числом')  
except NegativeAgeError as e:  
    print(e)
```

Формат ввода 1:

Введите свой
возраст:
-9

Формат вывода 1:

Возраст не может
быть отрицательным

Пользовательские исключения

Например:

```
class NegativeAgeError(Exception):  
    pass  
  
try:  
    print('Введите свой возраст:')  
    age = int(input())  
    if age < 0:  
        raise NegativeAgeError('Возраст не может  
                                быть отрицательным')  
    print('Ваш возраст равен', age)  
except ValueError:  
    print('Возраст должен быть числом')  
except NegativeAgeError as e:  
    print(e)
```

Формат ввода 2:

Введите свой

возраст:

тридцать

Формат вывода 2:

Возраст должен

быть числом

Пользовательские исключения

Выберите корректный способ создания пользовательского исключения CustomError, наследуемого от Exception.

```
class CustomError():  
    pass
```

1

```
class CustomError(Exception):  
    pass
```

2

```
class CustomError(Exception):  
    return
```

3

```
def CustomError(Exception):  
    pass
```

4

```
CustomError(Exception) class:  
    pass
```

5

Пользовательские исключения

Что покажет приведенный ниже код?

```
class NumberNotInRangeError(Exception):  
    pass  
  
try:  
    number = int('3999')  
    if not 4000 < number < 8000:  
        raise NumberNotInRangeError('Число из  
недопустимого  
диапазона')  
    print(number)  
except NumberNotInRangeError as e:  
    print(e)
```

Результат:

Число из недопустимого диапазона